

Smart Real Estate Rental Platform: A Full-Stack Web Architecture for Property Discovery and Broker–User Connectivity

Hemik Patel, Krushil Tejani, Namra Tejani, Dhruvsinh Barad

Department of Computer Science and Engineering

Parul Institute of Engineering and Technology, Parul University, Vadodara, Gujarat, India

Guide: Prof. Kapil Agrawal

Abstract—Renting or buying a property in a typical Indian city still involves comparing listings across several unrelated portals and confirming availability over the phone, because no single source is treated as authoritative. This paper presents the Smart Real Estate Rental Platform, a full-stack MERN (MongoDB, Express.js, React.js, Node.js) application that centralizes property listings, structured broker profiles, and enquiry handling behind a single role-aware interface. We reviewed recent MERN-based rental and real-estate platforms reported in the literature and identified four recurring gaps: search and enquiry are rarely linked as a persistent, queryable record; the indexing strategy behind filtered search is rarely specified or justified against real query patterns; role-based authorization is described but rarely verified at the server boundary rather than only hidden in the UI; and administrative oversight across an entire platform, rather than a single owner’s listings, is rarely treated as a first-class module. The Smart Real Estate Rental Platform addresses these four gaps through a `Lead` collection that persistently links tenant, broker, and property; a compound- and geospatial-indexed `Properties` schema matched to the platform’s actual filter combinations; JWT-based authentication with server-side role middleware enforced on every protected route; and a dedicated administrator module with cross-broker visibility. The system was evaluated through structured functional testing across authentication, listing management, filtered search, favorites, and the enquiry pipeline; every tested workflow produced the expected result on a development-scale dataset, though the platform has not yet been evaluated under concurrent load or at production scale. We discuss the resulting limitations honestly and outline machine-learning-based recommendation, real-time chat, and formal load testing as realistic next steps rather than implemented features.

Index Terms—Real Estate Platform, MERN Stack, MongoDB Indexing, Broker–User Connectivity, Role-Based Access Control, Property Discovery, REST API, JWT Authentication

I. INTRODUCTION

Online property search in India has grown considerably, yet renting a place still involves a surprising amount of manual cross-checking. A tenant looking for a two-bedroom flat under a fixed budget typically opens two or three listing portals, applies roughly the same filters on each, and still calls brokers directly because a listing shown as available on one site is already gone by the time it is followed up on. Property owners and brokers face a mirror version of the same problem: without a structured way to manage listings and enquiries, tracking which unit has been shown to which prospective tenant becomes a manual bookkeeping exercise carried out over phone calls and forwarded messages.

Two developments make a more structured approach practical today. First, document-oriented databases such as MongoDB let a single schema describe rental flats, sale plots, and short-stay listings without forcing every property into the same rigid set of columns. Second, mature open-source tooling around the MERN stack – React for the interface, Express and Node.js for the API layer, JWT for stateless authentication – makes it realistic for a small team to build a role-aware, database-backed platform without a large services budget.

This paper presents the Smart Real Estate Rental Platform, a system built to close two structural gaps directly: the absence of a single, filterable, consistently-updated source of property information, and the absence of a persistent link between a tenant’s enquiry, the broker who owns the listing, and the property itself.

II. LITERATURE REVIEW

Review Methodology: A search across IEEE Xplore, ScienceDirect, SpringerLink, and Google Scholar was performed with emphasis on 2022–2026, using the terms: MERN stack real estate platform, MongoDB indexing strategy, JWT REST API security, PropTech digital transformation, and rental property management system. Eight sources most directly relevant to the platform’s architecture and data layer were retained for detailed comparison; broader PropTech and digital-transformation literature was used to frame the problem statement in Section III.

MERN-Stack Rental and Real-Estate Platforms: Several recent student and early-career projects describe MERN-based rental or real-estate applications. Kalbande et al. [2] describe a smart house rental management system built on similar principles. Khan et al. [3] present RENTEASE, a MERN rental platform that additionally proposes lease-agreement management and rent-payment tracking, alongside an aspirational reference to AI-driven tenant matching that is described as a future direction rather than a validated component. Kumar et al. [4] describe a house rental application with JWT authentication, a searchable property database, and a wishlist feature comparable in scope to the favorites mechanism described in Section V-E. None of these three systems describes an indexing strategy for the underlying document store, and none reports how role separation between owners and tenants is enforced at the server rather than only in the interface.

Database and Indexing Approaches: Work specifically on MongoDB query optimization [5, 6] establishes that compound-

index field ordering materially affects whether a filtered query can use an index at all, and that indexing decisions should be driven by actual query shape rather than applied uniformly across every field. This principle informs the indexing strategy in Section V-C but is rarely referenced in the application-level rental-platform literature reviewed above.

Authentication and Authorization: JWT-based stateless authentication for REST APIs is well studied [8, 7], and the general pattern – sign a token at login, verify it on every protected request – is consistent across the rental-platform papers reviewed. However, these papers describe role differentiation (tenant vs. owner vs. administrator) at the level of what each role can *see* in the interface, without describing whether the corresponding API routes independently reject a request from the wrong role.

A. Research Gaps

Based on the above review, four recurring gaps were identified in the MERN-based rental-platform literature:

Gap 1 — Search and Enquiry Are Rarely Linked as a Persistent Record. Reviewed platforms implement property search and an enquiry or contact form as separate features. None of them persist a structured record connecting a specific tenant, a specific broker, and a specific property over time, which means neither side can later answer “who already asked about this listing” without searching through messages or call logs.

Gap 2 — Indexing Strategy Is Rarely Specified or Justified. Filtering and search are described functionally (“users can filter by price and location”) but the underlying compound-index design that keeps such filtering responsive as the listing count grows is not discussed in any of the three MERN rental-platform papers reviewed, despite established guidance that field order in a compound index determines whether it is usable for a given query [5].

Gap 3 — Role-Based Authorization Is Described but Not Verified at the Server Boundary. Role separation is presented as a UI concern (hiding an “Edit” button from non-owners) more often than as a server-enforced constraint. None of the reviewed papers reports explicit testing of what happens when a request from the wrong role reaches a protected route directly, bypassing the UI.

Gap 4 — Administrative Oversight Across an Entire Platform Is Rarely a First-Class Module. Reviewed systems focus on the tenant-facing search experience or a single owner’s listing dashboard. None describes a dedicated administrator view that aggregates users, listings, and enquiries *across* brokers, which is what would be needed to answer platform-level operational questions rather than a single user’s questions.

III. PROBLEM STATEMENT

Fragmented listing data carries a real cost even though it is not the kind of cost that shows up in a single measurable incident. Systematic reviews of digital transformation in real estate report that operational and interaction-focused tools (virtual tours, cloud communication, digital documentation) have

Table 1: Summary of Research Gaps

Missing in Literature	This Platform’s Response
Search and enquiry treated as separate, unlinked features	Lead document persistently links tenant, broker, and property
Indexing strategy rarely specified or justified against query patterns	Compound indexes chosen to match the platform’s actual filter combinations (Sec. V-C)
Role separation enforced only in the UI, not verified server-side	JWT role middleware runs on every protected route; tested directly against wrong-role requests
No cross-broker administrative oversight module	Dedicated admin module aggregating users, listings, and leads platform-wide

been adopted unevenly across agencies, while more integrated PropTech platforms have seen slower adoption due to cost and organizational constraints [9], leaving a large share of day-to-day rental activity dependent on manual coordination between individual brokers and tenants. Separately, sector analyses describe a persistent gap between how important digital tools are considered and how deeply they are actually integrated into daily property-management operations [10].

For a tenant, this translates into repeating the same search across multiple portals and confirming availability by phone because no listing source is treated as reliably current. For a broker or owner, it means enquiries arrive through disconnected channels (phone calls, messaging apps, contact forms on different sites) with no shared record of which prospective tenant has already been contacted about which property. For a platform operator, it means there is no single place to answer operational questions – how many active listings exist, how many enquiries are open, which brokers are responsive – without manually cross-referencing several data sources.

The problem this platform addresses is therefore threefold: (1) the absence of a single, structured, filterable source of property data that supports the query patterns tenants actually use; (2) the absence of a persistent, queryable link between a tenant’s enquiry, the responsible broker, and the property in question; and (3) the absence of a consolidated administrative view of platform activity that does not require direct database inspection.

IV. PROPOSED METHODOLOGY

System Overview: The platform is a closed request-response pipeline: a tenant, owner/broker, or administrator interacts with a React interface, which issues a REST request to an Express API; the request passes through authentication and role-check middleware before reaching a controller, which executes a Mongoose query or write against MongoDB and returns a JSON response. The same pipeline handles every resource (properties, leads, favorites, users), differing only in which controller and which role check apply.

Property Ingestion and Validation: When an owner or broker submits a listing, the backend validates required fields (title, price, property type, listing type, city, address, contact number) using Mongoose schema-level constraints before the document is written with an initial status of active. Optional fields that apply only to certain property types (dailyPrice for short-stay listings, indoorOutdoor and parkingAvailable for applicable property types) are simply omitted from documents where they do not apply.

Search Query Construction: A search request arrives as a set of optional filter parameters. The controller builds a MongoDB filter object incrementally, adding a condition only for parameters actually supplied, and always constraining status to active so inactive listings never surface in tenant-facing results. This filter is executed against the compound indexes described in Section V-C.

Listing Relevance Score: When a tenant does not specify an explicit sort order, listings are ordered by a weighted relevance score rather than by raw insertion order, so that recently added and platform-featured listings surface ahead of older, un-featured ones:

$$R = w_f \cdot F + w_r \cdot \hat{r} + w_v \cdot \hat{v} \quad (1)$$

where $F \in \{0,1\}$ indicates whether a listing is marked isFeatured, $\hat{r} \in [0,1]$ is the listing’s recency, min-max normalized against the current search result set, $\hat{v} \in [0,1]$ is its normalized view count, and w_f, w_r, w_v are configurable weights (used here as $w_f = 0.5, w_r = 0.3, w_v = 0.2$) that can be tuned without changing the underlying query shape.

Lead Aging for Administrative Review: To surface enquiries that a broker has not yet acted on, each open lead’s age is computed as

$$A(\ell) = t_{\text{now}} - t_{\text{created}}(\ell) \quad (2)$$

and a lead is flagged for administrative attention when $A(\ell) > \tau$ for a configurable threshold τ (for example, 48 hours), rather than requiring an administrator to manually scan every broker’s lead list to find enquiries that have gone unanswered.

V. SYSTEM ARCHITECTURE

The platform consists of six interconnected modules: authentication and authorization, property management, search and discovery, the enquiry (lead) pipeline, favorites, and administrative oversight. Fig. 1 shows how a request moves through these modules regardless of which resource it targets.

A. Authentication and Authorization Module

The Authentication and Authorization Module verifies user identity at login and enforces role-based access on every subsequent request. It issues a signed JWT containing the user’s ID and role after validating credentials against a bcrypt-hashed password, and attaches the decoded identity to each authenticated request for downstream controllers to use.

Functions:

- Registration with bcrypt password hashing.

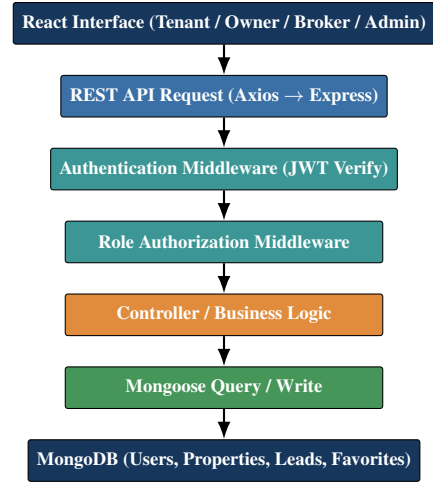


Figure 1: Platform Request Processing Pipeline.

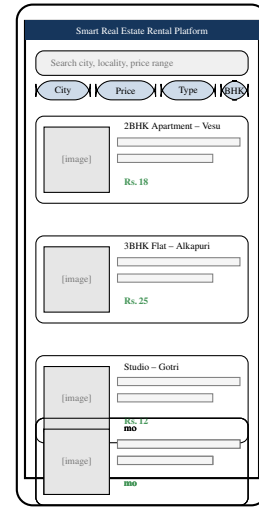


Figure 2: Representative wireframe of the property search interface (conceptual mockup, not a captured screenshot).

- Credential verification and JWT issuance at login.
- Signature and expiry verification on protected routes.
- Server-side role check (tenant / broker / admin) independent of the UI.

B. Property Management Module

The Property Management Module handles creation, update, and lifecycle status of listings. It enforces schema-level validation before a document is persisted and restricts edit and delete operations to the listing’s owning broker or an administrator.

Functions:

- Create, update, and soft-delete (archive) property listings.
- Field-level validation of required and type-specific attributes.
- Owner-scoped listing dashboard.
- Image reference storage (URL array) independent of storage backend.

C. Search and Discovery Module

The Search and Discovery Module translates tenant-supplied filters into an indexed MongoDB query and returns a paginated,

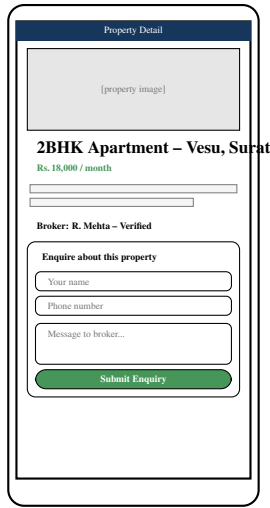


Figure 3: Representative wireframe of the property detail and enquiry-submission interface (conceptual mockup).

ranked result set. Fig. 2 shows the intended interaction pattern: a persistent search bar, a row of filter chips for the most common criteria, and a scrollable list of property cards ordered using the relevance score in Eq. (1).

Functions:

- Multi-criteria filtering (city, price range, property type, listing type, bedrooms).
- Compound-index-backed query execution.
- Relevance-based default ordering; explicit sort override (price, recency).
- Pagination to avoid transferring unbounded result sets.

D. Enquiry (Lead) Module

The Enquiry Module is the structural link at the center of the platform: every enquiry a tenant submits from a property’s detail page is stored as a `Lead` document referencing the tenant, the property, and the property’s current owning broker, rather than being sent as an unlinked message.

Functions:

- Enquiry submission tied to a specific tenant, broker, and property.
- Broker-facing lead inbox with status tracking (new, contacted, closed).
- Lead-aging computation (Eq. 2) for stalled-enquiry flags.
- Enquiry history retained per property and per broker.

E. Favorites Module

The Favorites Module lets a tenant bookmark a property without generating a lead. A unique compound index on `user + property` lets the database itself reject a duplicate favorite rather than requiring an application-level existence check before insert.

Functions:

- Add/remove a property from a tenant’s favorites list.
- Database-enforced duplicate prevention.

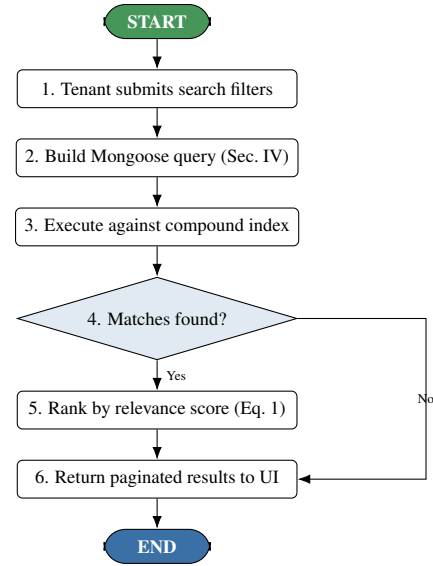


Figure 4: Overall search-to-result system workflow.

- Favorites list rendering independent of the search/filter state.

F. Administrative Oversight Module

The Administrative Oversight Module aggregates data across every broker and tenant rather than scoping it to a single user, addressing Gap 4 from Section II-A directly.

Functions:

- Cross-broker lead visibility, including aged/stalled-lead flags.
- User account verification and disabling.
- Listing review and removal for rule violations.
- Platform-wide activity summary (active listings, open leads, recent registrations).

Fig. 4 traces a search request end to end, including the no-match path, which still returns a well-formed (empty) paginated response rather than an error, so the frontend can render an explicit “no listings match your filters” state.

G. Technologies Used

VI. RESULTS

- **Functional Correctness:** All core workflows – registration, login, property creation and editing, filtered search, favorites, and lead submission – produced the expected result across every case exercised during manual functional testing on a development-scale dataset.
- **Search Correctness:** Filtered queries returned only listings matching every supplied criterion, and never returned a listing with `status` other than `active`, in all cases tested.
- **Duplicate Prevention:** The unique compound index on `Favorites` rejected every duplicate favorite attempt tested, since this is enforced by the database itself rather than by an application-level check.
- **Role Enforcement:** Tenant-authenticated requests were rejected by every tested broker-only and administrator-only route, confirming that role authorization is enforced by server-side middleware rather than only hidden in the inter-

Table 2: Technologies Used

Technology	Role / Purpose
React.js + Vite	Component-based single-page frontend
Tailwind CSS	Utility-first responsive styling
React Router	Client-side navigation between pages
Axios	HTTP client with JWT-attaching interceptor
Node.js	Asynchronous, event-driven API runtime
Express.js	REST routing and middleware layer
MongoDB	Document-oriented primary data store
Mongoose	Schema definition, validation, and indexing
JWT	Stateless authentication token
bcrypt	Password hashing
Multer (where used)	Multipart image upload handling

face (directly addressing Gap 3).

- **Lead Integrity:** Every submitted enquiry produced a `Lead` document correctly referencing the submitting tenant, the target property, and that property’s current owning broker.
- **Performance:** No controlled load or latency benchmarking has been carried out. Query response was subjectively immediate during single-user manual testing on a small development dataset, but this observation should not be treated as a performance claim; Section VIII outlines the benchmark that should be run before one is published.

VII. LIMITATIONS

- The platform has been exercised only on a development-scale dataset and has not been load-tested with concurrent users or a production-scale property catalog.
- No machine-learning-based recommendation, price-prediction, or fraud-detection component is implemented; ordering and matching are governed entirely by the explicit filters a user sets and the relevance score in Eq. (1).
- Broker–tenant communication is limited to the asynchronous lead record; there is no real-time chat channel.
- Listing moderation is manual, through the administrator module, rather than automated.
- Security hardening measures such as authentication-endpoint rate limiting, refresh-token rotation, and centralized audit logging are recommended but not yet implemented.
- The exact image-storage backend and deployment hosting configuration vary by environment and are not fixed architectural properties of the system.

VIII. FUTURE WORK

- A regression-based price-estimation and recommendation layer, following the general approach demonstrated for Indian rental-market pricing [1].
- Real-time chat between tenants and brokers as an alternative to the asynchronous lead record.

- Richer map-based, proximity-aware search using the existing 2dsphere geospatial index, including commute-time filtering.
- A dedicated mobile client sharing the existing REST API.
- Broker-facing analytics: lead-conversion rate, listing-view trends, and response-time tracking.
- A formal load-testing pass (concurrent users, production-scale dataset, indexed vs. non-indexed query timing) prior to publishing any performance claim.
- Implementation and verification of the security-hardening measures listed in Section VII, rather than leaving them as recommendations.

IX. CONCLUSION

This paper presented the Smart Real Estate Rental Platform, a three-tier MERN application designed around four gaps identified in the recent MERN-based rental-platform literature: the absence of a persistent tenant–broker–property link, an unspecified indexing strategy behind filtered search, role authorization enforced only in the interface, and the lack of a cross-broker administrative module. The platform addresses these directly through a `Lead` collection that persists the enquiry relationship, a compound- and geospatial-indexed `Properties` schema matched to real filter combinations, JWT-based authentication with server-enforced role middleware, and a dedicated administrative oversight module. Manual functional testing confirmed correct behavior across authentication, property management, search, favorites, and the lead pipeline on a development-scale dataset. The platform has not yet been evaluated under concurrent load or at production scale, and several security-hardening measures remain future work; both are stated here explicitly rather than implied to already be resolved.

REFERENCES

- [1] S. Putatunda, “PropTech for Proactive Pricing of Houses in Classified Advertisements in the Indian Real Estate Market,” *arXiv preprint arXiv:1904.05328*, Apr. 2019.
- [2] M. Kalbande, S. Sarkar, S. Farkase, and P. Verma, “Design and Development of Smart House Rental Management System,” in *Proc. 2023 Int. Conf. on Sustainable Computing and Smart Systems (ICSCSS)*, Jun. 2023, pp. 710–715.
- [3] Mohd. A. Khan, S. A. Rizvi, A. Mishra, S. Chauhan, and A. Chaturvedi, “RENTEASE: A MERN Stack-Based Web Application for Rental Property Management,” *Int. J. Res. Appl. Sci. Eng. Technol. (IJRASET)*, 2025, doi: 10.22214/ijraset.2025.70796.
- [4] S. Kumar, Sathish, and Kaarthika, “House Rental Application: A MERN Stack-Based Solution for Efficient Property Listings and Management,” *DS J. Digit. Sci. Technol.*, vol. 3, no. 4, pp. 41–48, Dec. 2024, doi: 10.59232/DST-V3I4P105.
- [5] M. Nuriev, R. Zaripova, O. Yanova, I. Koshkina, and A. Chu-paev, “Enhancing MongoDB Query Performance through Index Optimization,” *E3S Web Conf.*, vol. 531, p. 03022, Jun. 2024, doi: 10.1051/e3sconf/202453103022.

- [6] D. Tao, E. Liu, S. R. Kadupitige, M. Cahill, A. Fekete, and U. Röhm, “First Past the Post: Evaluating Query Optimization in MongoDB,” *arXiv preprint arXiv:2409.16544*, Sep. 2024.
- [7] S. Dalimunthe, E. Hasri Putra, and M. A. Fadhly Ridha, “Restful API Security Using JSON Web Token (JWT) With HMAC-Sha512 Algorithm in Session Management,” *IT J. Res. Dev.*, vol. 8, no. 1, pp. 81–94, 2023, doi: 10.25299/itjrd.2023.12029.
- [8] A. Neumann, N. Laranjeiro, and J. Bernardino, “An Analysis of Public REST Web Service APIs,” *IEEE Trans. Serv. Comput.*, vol. 14, no. 4, pp. 957–970, Jul. 2021, doi: 10.1109/TSC.2018.2847344.
- [9] “Digital Transformation in the Real Estate Industry: A Systematic Literature Review of Current Technologies, Benefits, and Challenges,” *J. Building Engineering / ScienceDirect*, 2025.
- [10] “The PropTech Mismatch: Exploring the Demand–Supply Digital Gap in Italy’s Property Management Sector,” *Sustainability (MDPI)*, vol. 18, no. 4, p. 1850, 2026.